

Virtual Environment Development

Design Lab 01

V. Markos

Mediterranean College

In this lab series we gradually build a (part) of the legendary *Zork I: The Great Underground Empire* map, focusing on both environment design and gameplay aspects. You can follow any part of these lab series either as stated here, or by properly mapping the instructions to your idea for your semester project. This week, we start by getting accustomed with the game, while creating a comprehensive map and layout of our environment.

Warm Up

Have you ever played any of the Zork series games? If not, then now it is the time to do so! You can find faithful revivals of the game on various places, such as <https://www.pcjs.org/software/pcx86/game/infocom/zork1/>. Take a handful of minutes to get accustomed to the game, its narrative, the environment and so on.

Exercise 1. Pick a part of the environment you found interesting and, by replaying¹ that part of the game, try to create a draft top-down view of the environment. Stay abstract, so feel free to just draw a conceptual map. Among others, you might consider a graph-based representation, i.e.:

- use *nodes* labelled by Zork-related names for the key places your environment is going to include, e.g., “Kitchen”, “Troll Room” and so on;
- use *directed edges* to connect the aforementioned nodes, also indicating the direction of movement, to ensure consistency with the original game.

In any case, your draft design should respect the topology implied by the original game!

After completing Exercise 1 you should have a rough idea of how the environment you are going to design should look at a really abstract level. This is really important as the work we are doing at the moment will serve as a blueprint when crafting the actual environment in the next steps. Speaking of next steps:

Exercise 2. Start by mapping the conceptual graph designed in Exercise 1 into an actual top-down map of the game. Use pen(cil) and paper or any drawing tool you enjoy most. Make sure that you follow the logical and spatial connections you spotted in your conceptual graph — or revise it accordingly, if needed. Moreover, this is the time to make some important decisions, as we pass from text to more concrete mappings:

- For example, in Zork, moving from the kitchen to the living room is “instant”. How are you going to separate those two rooms?
 - Are they *adjacent*, i.e., sharing a wall?

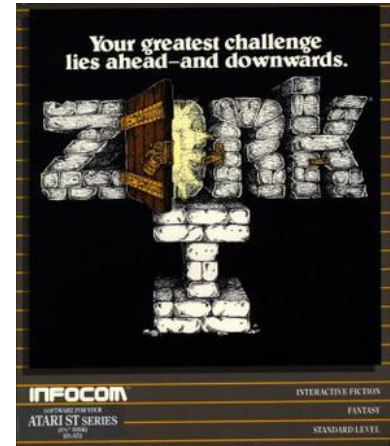


Figure 1: Zork I cover art. Even at the game’s cover, there are hints about the game and its plot, indicating a wildly interwoven game design. Source: <https://en.wikipedia.org/wiki/Zork>.

¹ Replaying the game to reach some deep underworld parts of the game or solving all riddles might be too much for a single lab session. So, you might take a nice shortcut by reading through the game transcript, found in assets/-ZORK_I_transcript.html

Back and forth is a commonality in the design process, especially at these early stages. So, feel free to swing a bit between your choices. However, too much usually means you should take time to think of your design choices again.

- Are they *proximate*, i.e., connect through a hallway or a closed door?

There is no correct answer, in principle, but the way an environment is designed does affect the way the game is played and experienced. So, take some time at this moment to reflect on choices like the above.

- *Scale* is also something to be considered at this point. How large is the “Main Hall”? How large is the “Kitchen”? What about the “Chasm”? Make concrete decisions on room sizes and make them visible on your map.
- *Verticality* is another thing that will often come up during this stage. Two dimensional mapping of vertical movement is, in general, implemented by layering the map and having some indicators² for upwards / downwards movement, e.g., ⊕ for downwards and ⊙ for upwards.

There have been a lot of spoilers so far, and more to come as we proceed with our labs, so, you might want to spend your next weekend exploring Zork unspoiled...

² ⊕ looks like a (physical) arrow’s feather tail seen from behind and, accordingly, ⊙ looks like an arrow’s point shot directly at us.

Layout

So far you should have a good understanding of the environment at an abstract level. The information gathered so far is enough to design a proper and complete layout diagram, which is something you might have almost or completely accomplished as you draft out your 2D map for Exercise 2.

Exercise 3. Using all you have noted and drafted in Section , create the layout diagram for the portion of Zork I’s environment you are going to design. Focus on the following aspects for your final design:

- Scale and sizing should be clearly reflected on your design, both by the design itself and as clear indications, e.g., through labels, or as a map legend.
- Labelling is your friend! Use labels to indicate entry points, connections between the various spaces and for anything you might feel needs further elaboration.
- Use some visual indicators for key elements of the environment, e.g., “grating”, or the “small brick building” and so on.

Using the above, you should deliver a full-fledged layout diagram, ready to be materialised using Unreal Engine!

Blockout

So far, we have a useful blueprint, but it’s time we got our hands dirty with Unreal Engine³! For this and any upcoming labs in this series, we will assume working on Unreal Engine (UE) 5.x, so make any necessary steps towards installing the appropriate version of the engine.

Exercise 4. First things first: open up your UE editor and create a new Blank or Third Person project. Then:

³ A really comprehensive guide on how to run Unreal Engine (minus some features we will not explore for a while) on really low-end machines: <https://vagon.io/blog/how-to-run-unreal-engine-on-a-low-end-device-even-without-gpu>.

Principle	Detail	Zork Example
Connectivity	Ensure movement between rooms is possible and matches the directional exits from the Zork map (e.g., North, Down).	A “North” exit from the Kitchen must lead immediately into the Living Room space.
Scale & Proportion	Doors must be high enough for the player to pass through; rooms must be proportional to their implied function (e.g., a “Crypt” is small; a “Main Hall” is large).	A 10m×10m room floor block. A doorway opening of 2.5m high by 1.5m wide.
Height and Verticality	Use the Z-axis correctly for “Up” and “Down” movements (e.g., placing the “Attic” floor high above the “Bedroom” floor).	A staircase or shaft geometry must connect the two floors, with the proper spatial separation.
Collision	Crucial: Test often. The player character must be able to walk through the environment without getting stuck or falling through the floor.	Run the game frequently to jump, walk, and verify all pathways work.

Table 1: Some common blocking principles you might find useful bearing in mind while blocking out your level.

- Introduce a Third Person Character Blueprint into the scene, or just use a Capsule (Cylinder) mesh set to 180 cm. This will serve as a scaling measure to judge environment scaling every time a modification is made, so, make sure to keep it visible!
- Spot the world’s origin, $O(0, 0, 0)$, which will serve as the place from where the first room of the level will be blocked out.

After a basic engine setup has been completed, the next step to actually add some models in the scene. There are primarily two ways of doing so in UE5:

- *Brush Actors (BSP)*, accessible from the *Geometry* tools. They offer an easier interface to interact and finely carve doorways on walls and such, through *Geometry Editing Mode*, so they are a good starting point to generate some block-outs quickly.
- Simple Static Meshes, which can be used to place basic models, e.g., from *Starter Content* or through *Modelling Tools*. The good thing with static meshes is that it is easy to replace with the actual asset meshes that will be used in the Greybox, so it might be a good idea to use them for your final version of the blockout.

Exercise 5. Start blocking out your level, also keeping in mind the design principles discussed in Table 1. One possible workflow might be to use BSP to create a few designs that you might consider as plausible and then, after any necessary modifications, use actual static meshes to create

the environment for your game. Do not focus that much on detail as on properly mapping the concepts from your layout diagram appropriately to the environment, maintaining the Zork feel as much as possible.